🔹 **Traditional metrics (you already know)**

- Throughput (requests/sec)
- Latency (response time)
- Concurrency handling

🔹 **New GenAI-specific metrics**

- **Time to First Token (TTFT)** → how fast response starts
- **Tokens/sec (generation speed)** → output speed
- **End-to-end latency** (prompt → final output)
- **Cost per request** (VERY important in LLM systems)
- **Model load handling** (GPU/CPU utilization)
- **Context window performance** (long prompts slow things down)

30-Day Roadmap: GenAI Performance Engineer

🎯 End Goal (what you'll achieve)

By Day 30, you will:

- Build a **GenAI app (chatbot / RAG)**
- Load test it using **modern tools**
- Measure **GenAI-specific performance metrics**
- Create a **portfolio project (very important for career shift)**

📅 WEEK 1: GenAI Foundations (Don't overdo theory)

🎯 **Goal:**

Understand how LLM systems behave from a performance lens.

📚 **Learn:**

- What is an LLM (tokens, context window, inference)
- Request lifecycle:
  `User → Prompt → Model → Tokens → Response`

🛠️ **Hands-on:**

- Use OpenAI API (or Hugging Face free models)
- Build a simple Python script:
  - Send prompt
  - Measure response time

👉 Track:

- Latency per request
- Output length (tokens)

💡 **Key Concepts to focus:**

- Tokens ≠ characters
- Larger prompt → higher latency
- Streaming vs full response

📅 WEEK 2: Bring in Performance Testing (your strength)

🎯 **Goal:**

Apply your NeoLoad mindset to GenAI APIs.

🛠️ **Tools to learn:**

- k6 (must-learn)
- Optional: Locust

🧪 **Hands-on Project:**

Load test your LLM API:

Test scenarios:

- 10, 50, 100 concurrent users
- Short vs long prompts
- Streaming vs non-streaming

📊 **Measure:**

- Response time
- Throughput
- Error rate

👉 NEW metrics (important):

- Time to First Token (TTFT)
- Tokens/sec

💡 **Insight you should discover:**

GenAI APIs degrade differently than normal APIs (non-linear latency)

📅 WEEK 3: Build a Real GenAI App (RAG system)

🎯 **Goal:**

Simulate real-world system (not just API calls)

🛠️ **Stack:**

- LangChain
- Vector DB (FAISS or similar)

🧪 **Build:**

A **RAG (Retrieval-Augmented Generation) chatbot**

Flow:

1. User query
2. Retrieve documents
3. Send context + prompt to LLM
4. Generate answer

📊 **Now test performance of:**

- Retrieval latency

- LLM latency
- End-to-end latency

🔍 **Introduce Observability:**

- LangSmith
- Helicone

💡 **Key learning:**

> Bottleneck is often NOT the model — it's retrieval + orchestration

---

📅 WEEK 4: Advanced Performance + Portfolio

🎯 **Goal:**

Differentiate yourself (this is where most people stop — you won't)

🔹 Part 1: Optimization Experiments

Test & document:

1. Prompt Optimization

- Short vs long prompts
- Structured prompts

2. Caching

- Repeated queries
- Response reuse

3. Model Comparison

- Small vs large models
- Cost vs latency tradeoff

🔹 Part 2: Inference Performance (optional but powerful)

Explore:

- vLLM
- TensorRT-LLM

(You don't need deep infra — just understand concepts)

🔹 Part 3: Portfolio Project (CRITICAL)

Build this:

"GenAI Performance Benchmarking Suite"

Include:

- Chatbot / RAG system
- Load testing scripts (k6)
- Performance report

📄 **Your report should include:**

- Latency vs concurrency graph
- Cost vs usage
- Bottleneck analysis
- Optimization suggestions

🧠 Daily Time Plan (Realistic)

Time	Activity
1–1.5 hr	Learning
1–2 hr	Hands-on
Weekend	Deep project work

🧭 What NOT to waste time on

❌ Deep ML math
❌ Building models from scratch
❌ Random GenAI tutorials

👉 Focus on:

System behavior + performance + scaling

🏁 Final Outcome (after 30 days)

You can confidently say:

> “I specialize in performance testing of GenAI systems — including LLM APIs, RAG pipelines, and inference optimization.”

That’s a **rare and high-value positioning**.